2a. Claude and gemini
2b. Claude through the website, and gemini through the website
2c. Prompt used:
“
Generate C++ code using objects, not functions, to implement the following requirements
• The program will prioritize emails for a busy company CEO.
• You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
• You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
• You must create objects from scratch. Do not include pre-existing heap modules.
• Test File format:
Command Description
EMAIL <sender category>,
<subject line>,
<date>
The emails in the CEO’s Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:
• Boss – read first
• Subordinate – read next
• Peer – read next
• ImportantPerson – read next
• OtherPerson – read last
If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.
EMAIL is followed by space. The rest of the fields are delimited.
Assume <sender category> is one of the five strings listed above.
Assume <subject line> is a string which may contain spaces, but not commas
Assume <date> is in the format: MM-DD-YYYY
NEXT
Next email for the CEO to read. Display the information on the terminal in the following format:
Sender:
Subject:
Date:
READ CEO has read email and dealt with it
COUNT Display current count of unread emails

• The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).
• Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

• Your program should display this:

```
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.
```

“

2d. Claude’s code
“

```
// CEO Email Prioritizer - MaxHeap (array/list-based) priority queue
// Build: g++ -std=c++11 -o ceo_email ceo_email.cpp
// Run:   ./ceo_email testfile.txt     (or ./ceo_email and type the file name)

#include <iostream>
#include <fstream>
#include <string>

using namespace std;

// ---------------------------------------------------------------------------
// Email: holds one message and knows how to compare its priority to another.
// ---------------------------------------------------------------------------
class Email {
private:
    string sender;
    string subject;
    string date;      // MM-DD-YYYY
    int    rank;      // higher = read sooner (Boss=5 ... OtherPerson=1)
    int    dateKey;   // YYYYMMDD, so bigger = newer
    long   sequence;  // arrival order, used only to break exact ties

    static int rankFor(const string& category) {
        if (category == "Boss")            return 5;
        if (category == "Subordinate")     return 4;
        if (category == "Peer")            return 3;
        if (category == "ImportantPerson") return 2;
        return 1;      // OtherPerson
    }

    static int dateKeyFor(const string& d) {
        // MM-DD-YYYY -> YYYYMMDD
        int mm = stoi(d.substr(0, 2));
        int dd = stoi(d.substr(3, 2));
        int yy = stoi(d.substr(6, 4));
        return yy * 10000 + mm * 100 + dd;
    }

public:
    Email() : rank(0), dateKey(0), sequence(0) {}

    Email(const string& s, const string& subj, const string& d, long seq)
```

```
        : sender(s), subject(subj), date(d),
          rank(rankFor(s)), dateKey(dateKeyFor(d)), sequence(seq) {}

    const string& getSender()  const { return sender; }
    const string& getSubject() const { return subject; }
    const string& getDate()    const { return date; }

    // True if this email should be read BEFORE the other one.
    bool higherPriorityThan(const Email& other) const {
        if (rank != other.rank)       return rank > other.rank;       // category first
        if (dateKey != other.dateKey) return dateKey > other.dateKey; // newest first
        return sequence < other.sequence;                             // earlier arrival
    }

    void display() const {
        cout << "Sender: "  << sender  << endl;
        cout << "Subject: " << subject << endl;
        cout << "Date: "    << date    << endl;
    }
};

// ---------------------------------------------------------------------------
// MaxHeap: list-based (dynamic array) binary heap built from scratch.
// ---------------------------------------------------------------------------
class MaxHeap {
private:
    Email* data;
    int    count;
    int    capacity;

    static int parent(int i) { return (i - 1) / 2; }
    static int left(int i)   { return 2 * i + 1; }
    static int right(int i)  { return 2 * i + 2; }

    void grow() {
        int newCap = capacity * 2;
        Email* bigger = new Email[newCap];
        for (int i = 0; i < count; i++) bigger[i] = data[i];
        delete[] data;
        data = bigger;
        capacity = newCap;
    }

    void swapAt(int a, int b) {
```

```
        Email tmp = data[a];
        data[a] = data[b];
        data[b] = tmp;
    }

    void siftUp(int i) {
        while (i > 0 && data[i].higherPriorityThan(data[parent(i)])) {
            swapAt(i, parent(i));
            i = parent(i);
        }
    }

    void siftDown(int i) {
        while (true) {
            int best = i;
            int l = left(i), r = right(i);
            if (l < count && data[l].higherPriorityThan(data[best])) best = l;
            if (r < count && data[r].higherPriorityThan(data[best])) best = r;
            if (best == i) break;
            swapAt(i, best);
            i = best;
        }
    }

public:
    MaxHeap() : data(new Email[16]), count(0), capacity(16) {}
    ~MaxHeap() { delete[] data; }

    // Prevent accidental shallow copies of the owned array.
    MaxHeap(const MaxHeap&) = delete;
    MaxHeap& operator=(const MaxHeap&) = delete;

    bool isEmpty() const { return count == 0; }
    int  size()    const { return count; }

    void insert(const Email& e) {
        if (count == capacity) grow();
        data[count] = e;
        siftUp(count);
        count++;
    }

    // Returns the highest-priority email without removing it.
    const Email& peekMax() const { return data[0]; }
```

```
    // Removes the highest-priority email.
    void removeMax() {
        if (count == 0) return;
        count--;
        data[0] = data[count];
        if (count > 0) siftDown(0);
    }
};

// ---------------------------------------------------------------------------
// CEOInbox: wraps the heap and implements the EMAIL / NEXT / READ / COUNT
// commands.
// ---------------------------------------------------------------------------
class CEOInbox {
private:
    MaxHeap heap;
    long    nextSequence;

    static string trim(const string& s) {
        size_t b = s.find_first_not_of(" \t\r\n");
        if (b == string::npos) return "";
        size_t e = s.find_last_not_of(" \t\r\n");
        return s.substr(b, e - b + 1);
    }

public:
    CEOInbox() : nextSequence(0) {}

    // arguments = everything after "EMAIL ", i.e. "<category>,<subject>,<date>"
    void addEmail(const string& arguments) {
        size_t c1 = arguments.find(',');
        size_t c2 = arguments.find(',', c1 == string::npos ? 0 : c1 + 1);
        if (c1 == string::npos || c2 == string::npos) return; // malformed

        string category = trim(arguments.substr(0, c1));
        string subject  = trim(arguments.substr(c1 + 1, c2 - c1 - 1));
        string date     = trim(arguments.substr(c2 + 1));

        heap.insert(Email(category, subject, date, nextSequence++));
    }

    void next() const {
        if (heap.isEmpty()) {
```

```
            cout << "No emails to read." << endl;
            return;
        }
        cout << "Next email:" << endl;
        heap.peekMax().display();
    }

    void read() {
        heap.removeMax(); // does nothing if empty
    }

    void count() const {
        cout << "There are " << heap.size() << " emails to read." << endl;
    }

    void processCommand(const string& rawLine) {
        string line = trim(rawLine);
        if (line.empty()) return;

        if (line.compare(0, 6, "EMAIL ") == 0) addEmail(line.substr(6));
        else if (line == "NEXT")              next();
        else if (line == "READ")              read();
        else if (line == "COUNT")               count();
    }
};

// ---------------------------------------------------------------------------
// Driver
// ---------------------------------------------------------------------------
int main(int argc, char* argv[]) {
    string filename;
    if (argc > 1) {
        filename = argv[1];
    } else {
        cout << "Enter test file name: ";
        getline(cin, filename);
    }

    ifstream in(filename.c_str());
    if (!in) {
        cerr << "Could not open file: " << filename << endl;
        return 1;
    }
```

```
    CEOInbox inbox;
    string line;
    while (getline(in, line)) {
        inbox.processCommand(line);
    }
    return 0;
}
```
“

2d. Gemini’s code

“

```
#include <iostream>
#include <string>
#include <sstream>

// Class representing an individual Email
class Email {
private:
    std::string sender;
    std::string subject;
    std::string date; // MM-DD-YYYY
    int senderPriority;
    int dateSortKey;  // YYYYMMDD for numerical comparison

    int calculateSenderPriority(const std::string& category) const {
        if (category == "Boss") return 5;
        if (category == "Subordinate") return 4;
```

```
        if (category == "Peer") return 3;
        if (category == "ImportantPerson") return 2;
        return 1; // OtherPerson
    }

    int parseDate(const std::string& d) const {
        if (d.length() < 10) return 0;
        int month = std::stoi(d.substr(0, 2));
        int day = std::stoi(d.substr(3, 2));
        int year = std::stoi(d.substr(6, 4));
        return year * 10000 + month * 100 + day;
    }

public:
    Email() : senderPriority(0), dateSortKey(0) {}

    Email(const std::string& s, const std::string& subj, const std::string& d)
        : sender(s), subject(subj), date(d) {
        senderPriority = calculateSenderPriority(s);
        dateSortKey = parseDate(d);
    }

    std::string getSender() const { return sender; }
    std::string getSubject() const { return subject; }
    std::string getDate() const { return date; }

    // Priority Comparison:
    // Primary key: Sender Category
    // Secondary key: Newest date first (higher dateSortKey)
    bool operator>(const Email& other) const {
        if (this->senderPriority != other.senderPriority) {
            return this->senderPriority > other.senderPriority;
        }
        return this->dateSortKey > other.dateSortKey;
    }

    bool operator<(const Email& other) const {
        if (this->senderPriority != other.senderPriority) {
            return this->senderPriority < other.senderPriority;
        }
        return this->dateSortKey < other.dateSortKey;
    }
};
```

```
// Node structure for Doubly Linked List implementation
struct Node {
    Email data;
    Node* prev;
    Node* next;

    Node(const Email& email) : data(email), prev(nullptr), next(nullptr) {}
};

// List-based MaxHeap Implementation
class ListMaxHeap {
private:
    Node* head;
    Node* tail;
    int heapSize;

    // Helper method to retrieve node at 1-based index
    Node* getNodeAt(int index) const {
        if (index < 1 || index > heapSize) return nullptr;

        Node* curr = head;
        for (int i = 1; i < index && curr != nullptr; ++i) {
            curr = curr->next;
        }
        return curr;
    }

    void swapNodeValues(Node* a, Node* b) {
        if (a && b) {
            Email temp = a->data;
            a->data = b->data;
            b->data = temp;
        }
    }

    void heapifyUp(int index) {
        while (index > 1) {
            int parentIndex = index / 2;
            Node* currNode = getNodeAt(index);
            Node* parentNode = getNodeAt(parentIndex);

            if (currNode && parentNode && currNode->data > parentNode->data) {
                swapNodeValues(currNode, parentNode);
                index = parentIndex;
```

```
            } else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        while (2 * index <= heapSize) {
            int leftIndex = 2 * index;
            int rightIndex = 2 * index + 1;
            int largestIndex = index;

            Node* largestNode = getNodeAt(largestIndex);
            Node* leftNode = getNodeAt(leftIndex);
            Node* rightNode = getNodeAt(rightIndex);

            if (leftNode && leftNode->data > largestNode->data) {
                largestIndex = leftIndex;
                largestNode = leftNode;
            }

            if (rightNode && rightNode->data > largestNode->data) {
                largestIndex = rightIndex;
                largestNode = rightNode;
            }

            if (largestIndex != index) {
                swapNodeValues(getNodeAt(index), largestNode);
                index = largestIndex;
            } else {
                break;
            }
        }
    }

public:
    ListMaxHeap() : head(nullptr), tail(nullptr), heapSize(0) {}

    ~ListMaxHeap() {
        Node* curr = head;
        while (curr) {
            Node* nextNode = curr->next;
            delete curr;
            curr = nextNode;
```

```
    }
}

void push(const Email& email) {
    Node* newNode = new Node(email);
    if (!head) {
        head = tail = newNode;
    } else {
        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode;
    }
    heapSize++;
    heapifyUp(heapSize);
}

Email peek() const {
    if (isEmpty()) {
        return Email();
    }
    return head->data;
}

void pop() {
    if (isEmpty()) return;

    if (heapSize == 1) {
        delete head;
        head = tail = nullptr;
    } else {
        // Swap head root with last node
        swapNodeValues(head, tail);

        // Remove tail node
        Node* oldTail = tail;
        tail = tail->prev;
        tail->next = nullptr;
        delete oldTail;

        // Re-heapify root
        heapifyDown(1);
    }
    heapSize--;
}
```

```
    bool isEmpty() const {
        return heapSize == 0;
    }

    int size() const {
        return heapSize;
    }
};

// Manager class handling CEO Inbox Queue
class CEOInbox {
private:
    ListMaxHeap priorityQueue;

    // Helper function to trim whitespace
    std::string trim(const std::string& str) {
        size_t first = str.find_first_not_of(" \t\r\n");
        if (first == std::string::npos) return "";
        size_t last = str.find_last_not_of(" \t\r\n");
        return str.substr(first, (last - first + 1));
    }

public:
    void processCommand(const std::string& line) {
        if (line.empty()) return;

        if (line.substr(0, 5) == "EMAIL") {
            std::string data = line.substr(6);
            std::stringstream ss(data);
            std::string sender, subject, date;

            std::getline(ss, sender, ',');
            std::getline(ss, subject, ',');
            std::getline(ss, date, ',');

            Email newEmail(trim(sender), trim(subject), trim(date));
            priorityQueue.push(newEmail);
        }
        else if (line == "NEXT") {
            if (!priorityQueue.isEmpty()) {
                Email top = priorityQueue.peek();
                std::cout << "Next email:\n";
                std::cout << "Sender: " << top.getSender() << "\n";
```

```
            std::cout << "Subject: " << top.getSubject() << "\n";
            std::cout << "Date: " << top.getDate() << "\n";
        }
    }
    else if (line == "READ") {
        if (!priorityQueue.isEmpty()) {
            priorityQueue.pop();
        }
    }
    else if (line == "COUNT") {
        std::cout << "There are " << priorityQueue.size() << " emails to read.\n";
    }
  }
};

int main() {
  CEOInbox inbox;
  std::string line;

  while (std::getline(std::cin, line)) {
    inbox.processCommand(line);
  }

  return 0;
}
“
```

2e. Individual Analysis

Claude

The time complexity is O(log n) for EMAIL and READ since the heap has to be traversed. Every step through filters out half the total emails. The time complexity for COUNT and NEXT are O(1), since they are just returning the size,m and reading data at position 0 respectively. The space complexity is O(n), there is a set amount of memory that each email takes up. Reading emails doesn’t decrease the memory allocated. The program runs correctly, but crashes on bad dates. The comments are alright, the lack of comments in max heap is the biggest area of concern.

Gemini

The time complexity is not the same as Claude’s code. EMAIL and read are instead O(n), since it is using a doubly linked list The space complexity is also once again O(n), with a set amount of memory for each email. Reading emails this time however does decrease the total number of memory used. It failed to follow instructions on taking an input file, and instead took command line input. The command line input can’t handle EMAIL commands without anything after, and requires commands to have no trailing whitespace. Comments are sparse and not in areas with

high complexity

Difference
Gemini’s code does not have a tie breaker, so two emails with similar categories and dates will end up in an order without any meaning. Gemini’s not taking an input file is a major problem, however claude not freeing memory upon reading the emails is a rather large problem too. Taking one as is with no improvements Claude’s would win out because it can take an input file. I will be choosing Gemini’s to improve on however, since the doubly linked list is a better approach since memory is freed once read. In a situation where this would operate continuously, freeing memory would be critical, since otherwise the program would eventually run out of memory. Pieces of Claude’s program are useable in Gemini’s with slight modification, such as reading the input file.

Improvements
Tie breaking, using line trimming, email requires a trailing space, next on empty has a print, decrement heap size before heapify down,pass in file on command line